

An Empirical Study on Estimating Energy Consumption From Software Traces

Phuong Nguyen
Université de Pau et des Pays de
l'Adour, E2S UPPA, LIUPPA
Pau, France
thi-mai-phuong.nguyen@univ-
pau.fr

Adel Nouredine
Sorbonne Université, CNRS, LIP6
Paris, France
Université Paris Nanterre
Paris, France
adel.nouredine@lip6.fr

Congduc Pham
Université de Pau et des Pays de
l'Adour, E2S UPPA, LIUPPA
Pau, France
congduc.pham@univ-pau.fr

Abstract

Although numerous hardware and software solutions exist for energy monitoring, they often require extensive setup, configuration, and technical expertise. This paper presents a preliminary study investigating the potential of widely available software traces to estimate software energy consumption. Experiments across synthetic and realistic workloads show that certain hardware metrics and structured software logs, such as SQL statements and Docker container logs, consistently correlate with energy usage, whereas unstructured logs, including general HTTP or Maven logs, exhibit less reliable correlations. These findings suggest that combining hardware metrics with structured software traces, while accounting for workload characteristics and application structure, offers a promising approach for estimating software energy consumption without dedicated energy monitoring tools.

CCS Concepts

• **Hardware** → Power estimation and optimization; • **Software and its engineering** → Software organization and properties.

Keywords

Software Traces, Energy Consumption, Software Logs, Software Energy

ACM Reference Format:

Phuong Nguyen, Adel Nouredine, and Congduc Pham. 2025. An Empirical Study on Estimating Energy Consumption From Software Traces. In *IEEE/ACM 12th International Conference on Big Data Computing, Applications and Technologies (BDCAT '25)*, December 01–04, 2025, Nantes, France. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3773276.3774295>

1 Introduction

The growing global demand for digital services has positioned Information and Communications Technologies (ICT) as a central driver of modern society. From personal devices and applications to large-scale data centers and cloud infrastructure, ICT systems underpin economic, social, and industrial activities worldwide.

This growth, however, comes with a substantial environmental cost. In 2020, ICT accounted for about 4% of the world's electricity

use and 1.4% of greenhouse gas emissions (GHGE) [10]. More recent reports highlight a worrying trend: by 2021, electricity use in the sector rose to 4.7%, and by 2022 its share of global GHGE had increased to 1.7% [3]. Estimates also indicate that ICT's global GHGE could exceed 14% in 2040 [4]. These figures reflect a rapid upward trajectory, raising concerns about sustainability and the long-term impacts of ICT on climate change.

As a result, the concept of Green Information Technology (Green IT) has become more recognized over time. Green IT includes different methods and practices designed to reduce energy consumption across all ICT layers, from hardware components to data centers and from applications to distributed systems. Although significant progress has been made in designing energy-aware software models that guide the development of new, efficient applications, it remains challenging to apply these practices to legacy software. Targeted optimization of energy hotspots is a more practical approach because large, long-standing codebases are often hard to refactor completely.

Addressing this challenge has significant implications. Many current IT systems still rely on legacy software that cannot be easily replaced. Enhancing energy efficiency in such systems, therefore, can lead to immediate and significant reductions in overall ICT energy demand. Consequently, the development of tools and methods that support energy-aware software engineering, even in the context of existing systems, is urgently needed.

Recent studies have proposed tools to monitor software energy consumption, such as PowerAPI [7] and PowerJoular [14]. These solutions provide valuable insights but introduce additional runtime overhead, which may in turn adversely affect system performance. This limitation naturally motivates the exploration of alternative resources. System and software logs, which are already generated in most systems and impose minimal performance impact, represent a promising but underutilized source of information. Logs have traditionally been used for debugging [28], anomaly detection [2], vulnerability identification [1, 5], and fault diagnosis [26, 27]. However, their potential to capture energy-related behavior has received little attention.

In this context, our study aims to extend research on energy-aware software by carefully examining the feasibility of using software logs to estimate energy consumption. We propose an empirical investigation into the relationship between logs, hardware metrics, and energy usage, thereby exploring a relatively underdeveloped but promising approach to lightweight and non-intrusive energy analysis. Accordingly, we pose the following key research question:

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only. Request permissions from owner/author(s).

BDCAT '25, Nantes, France

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2286-8/25/12
<https://doi.org/10.1145/3773276.3774295>

Can software energy consumption be estimated using software traces and logs?

To answer this question, we conduct an empirical study on synthetic and real workloads while collecting both hardware and software traces. Our objective is to determine whether software logs provide meaningful signals for tracing and estimating energy consumption, thus opening a new direction for energy-aware software engineering.

The remainder of this paper is organized as follows: Section 2 reviews related work, Section 3 describes the empirical methodology, Section 4 presents our results, Section 5 discusses their implications, and Section 7 concludes with future research directions.

2 Related Work

Over the past decade, the amount of research on software energy consumption has increased, notably, driven by growing concerns about the environmental impact of ICT systems. While early studies mostly focused on hardware-level optimization, recent work has shifted attention to the software domain, where design decisions, implementation choices, and runtime behavior can all have significant influences on energy use. As software complexity and scale continue to grow, it is essential to understand these factors in order to develop energy-aware practices and tools.

A significant quantity of research has examined how program characteristics interact with hardware features to affect energy consumption. Rajan et al. [23] demonstrated that execution time, cache access, memory instructions, context switches, and CPU migrations are all strongly correlated with energy use. Many of these factors also impact performance indicators, such as runtime and latency, underscoring the challenge of balancing performance optimization with energy efficiency. Their findings suggest that meaningful energy savings will depend on techniques capable of reducing both power and execution time simultaneously. Building on this perspective, Oliveira et al. [17] investigated the efficiency of common Java collections and found that widely used implementations such as `ArrayList`, `HashMap`, and `HashTable` are not always optimal from an energy standpoint. Similarly, Oumani et al. [20] showed that the choice of Java I/O libraries can substantially alter energy footprints. At the same time, subsequent work by the same authors demonstrated that Java Virtual Machine (JVM) configurations also play a critical role in determining energy use [18]. Beyond libraries and runtime settings, Martinez et al. [12] highlighted a trade-off between energy consumption and correctness in automatic program repair, and Nouredine et al. [16] explored potential links between code coverage metrics and energy usage, though their findings were inconclusive. In brief, these works illustrate the multifaceted nature of software energy consumption, where decisions made at the levels of code, libraries, runtime environments, and testing strategies all have significant implications. However, their applicability to legacy systems is limited, since extensive refactoring is often impractical.

Complementing these investigations, another line of research has focused on tools for measuring and analyzing software energy consumption. Studies with developers, one of its motivating factors, reveal that the lack of accessible and actionable tools remains a

barrier to wider adoption of energy-aware practices. For instance, Pinto et al. [22] confirmed that the lack of tool support poses significant hurdles to energy-aware software development. Ournani et al. [19], afterwards, reported that practitioners seek tools capable not only of measuring overall energy use but also of identifying root causes and suggesting effective optimizations. In consequence, several systems have been proposed to address these needs, such as `PowerJoular`, a cross-platform monitoring framework, along with `JoularJX`, a Java agent capable of tracking method-level energy consumption in real time [15]; or `PowerAPI`, a toolkit that supports software-defined power meters to monitor power consumption at the level of applications, containers, and code blocks [7]. While these tools eliminate the need for physical meters and offer valuable insights, they require installation, configuration, and integration into workflows, often introducing runtime overhead that limits their usability in production environments. These limitations motivate the search for alternatives that minimize performance impact while still providing meaningful energy insights.

An emerging opportunity lies in the use of software logs and traces. Logs are widely generated in most software systems and have long been leveraged for tasks such as debugging [28], anomaly detection [2], vulnerability identification [1, 5], and fault diagnosis [26, 27]. Recent advances have expanded their role: for instance, Formal Concept Analysis has been applied to log messages to help operators diagnose failures [8], while other approaches have focused on automatically detecting anomalous event sequences [11]. Peccia et al. [21] examined log characteristics that improve fault detection accuracy, and He et al. [9] systematically compared anomaly detection methods for log-based analysis. Logs have also proven effective for diagnosing performance issues [13], underscoring their versatility as a diagnostic resource. Given that both hardware and software characteristics directly influence energy use [23], logs offer a promising but largely unexplored avenue for energy analysis. By identifying correlations between log events and energy consumption, it may be possible to detect energy hotspots and anomalies without relying on additional monitoring frameworks.

In summary, prior research has established that software energy consumption is influenced by various factors. Based on this, researchers have also developed many tools to monitor and analyze energy usage with different levels of detail. However, these approaches often require invasive measurement equipment or impose overhead that limits their practicality. In contrast, logs are lightweight, widely available, and have already been embedded in most software systems, but their potential for energy estimation remains underexplored. In order to fill this gap and open a path towards practical, low-overhead solutions for energy-aware software engineering, this study investigates whether software energy consumption can be estimated from software traces and logs.

3 Experimental Methodology

Traces and logs constitute a heterogeneous and large-scale data source. To address the research question of this study, we designed a series of experiments aimed at identifying correlations between collected software traces, system logs, and energy consumption.

The methodology consists of two phases: (i) data collection during the execution of experimental workloads, and (ii) analysis of

the collected data. To enhance the reliability of the results, each workload was executed between 15 and 30 times, ensuring both the stability of the measurements and the accuracy of the subsequent analysis. An overview of the proposed methodology is presented in Fig. 1, which illustrates the process followed to address the research question of this study.

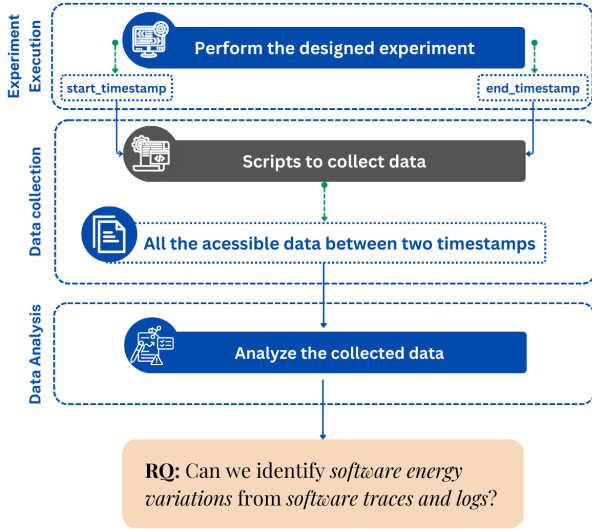


Figure 1: Methodology used to answer the research question.

The primary objective of this experimental design is to evaluate the feasibility and insight potential of using software logs and traces to estimate system energy consumption, rather than proposing a new modeling approach. Our experimental approach and design are built with replication in mind to guarantee transparent and reproducible analysis.

3.1 Experimental Workloads

To capture the diversity of traces and logs, we designed a comprehensive set of experiments aimed at collecting representative and varied data. These experiments are organized into two categories: (i) synthetic workloads generated using standard benchmarking tools, and (ii) realistic workloads executed using commonly used benchmark applications, DayTrader and Hotel Reservation, together with Maven, supplemented by a custom Java application developed for this study, referred to as *Custom REST App*.

3.1.1 Synthetic Workloads. Synthetic workloads are controlled tests designed to stress hardware components in isolation. They are standardized and repeatable, making them suitable for analyzing system behavior under well-defined conditions. In this study, we relied on the PassMark PerformanceTest benchmark tool for Linux¹ to generate three types of workloads: (i) CPU-intensive, (ii) memory-intensive, and (iii) combined CPU and memory-intensive tasks. The CPU-intensive workload includes operations such as integer and

floating-point arithmetic, prime number calculations, sorting, encryption, compression, physics simulations, and extended instruction sets (e.g., SSE, AVX). The memory-intensive workload consists of cached and uncached memory reads, memory writes, SQL database operations, latency measurements, and multi-threaded memory access. The combined workload executes both suites sequentially, first the CPU-intensive workload and then the memory-intensive workload, stressing computational and memory resources in succession.

For all experiments, the number of test processes and iterations was fixed at one ($-p\ 1\ -t\ 1$), and the test duration was configured as *very long* ($-d\ 4$). We assessed the consistency of the results by collecting metrics and energy measurements at two intervals: 100 milliseconds and 1 second, to evaluate the effect of sampling intervals on measurement consistency. This involved executing each workload 30 times with metrics collected at the 100-millisecond interval, and an additional 30 times with metrics collected at the 1-second interval.

3.1.2 Realistic Workloads. Realistic workloads involve actual applications or workflows, reflecting system behavior under practical usage scenarios. We selected four applications representing typical realistic workloads: the DayTrader benchmark, the microservice-based application Hotel Reservation, the Custom REST App, and Maven. Each workload was subjected to one or more experiments, which are described in detail in the following sections.

We repeated each experiment 15 times (except 30 times for the Maven workloads) to collect stable and reliable data despite variability introduced by background processes and operating system activity.

DayTrader. Daytrader is an open-source benchmark application that emulates an online stock trading system, allowing users to log in, view portfolios, access stock quotes, and perform stock transactions. In our study, we use two versions: Daytrader 7² with Java EE 7, and Daytrader 10³ with Jakarta EE 10.

We conducted the experiments using the provided JMX files, which define realistic user interaction scenarios with the application. These workloads simulate common trading activities, including logging in, retrieving quotes, viewing account information, performing buy or sell operations, and so on.

For both DayTrader 7 and DayTrader 10, we configured 50 virtual users ($THREADS = 50$) with no ramp-up delay ($RAMP = 0$), starting simultaneously and running continuously for 180 seconds ($DURATION = 180$), following the configuration provided by the DayTrader benchmark. The test plans contain a Random Controller to add variety to each virtual user, ensuring that no two users behave in the same way. The main difference between the two versions is the addition of JAX-RS requests in DayTrader 10.

In the DayTrader 7 scenario, the Random Controller selects between two Throughput Controller: one for JSF requests, executed roughly 10% of the time per virtual user, and one for JSP requests, executed approximately 90% of the time per virtual user.

¹https://www.passmark.com/products/pt_linux/

²<https://github.com/WASdev/sample.daytrader7>

³<https://github.com/OpenLiberty/sample.daytrader10>

For DayTrader 10, the Random Controller directs traffic among three Throughput Controller, with per-user execution frequencies of 10% for JSF requests, 70% for JSP requests, and 20% for JAX-RS requests.

Hotel Reservation. Hotel Reservation⁴ is a microservice-based application built with Go and gRPC that implements a hotel booking service, allowing users to simulate realistic usage scenarios.

Similar to Daytrader, we conducted the experiment using a JMX file derived from the provided workload definitions. The test plan was configured with 50 virtual users (THREADS = 50), ramped up within 1 second (RAMP = 1) and sustained for 200 seconds (DURATION = 200), roughly matching the runtime used for DayTrader to ensure comparable measurement conditions. Each virtual user repeatedly performed standard hotel booking workflows. During the 200-second test, users can search for hotels, request recommendations, log in, and make reservations. Any request failures were ignored to ensure the uninterrupted execution of the workflow.

Custom REST App. We developed a Java Client/Server application, referred to as the Custom REST App, that provides RESTful APIs for managing employee information, including adding, editing, and deleting records in a database. Each employee record includes an ID, first name, last name, and email. Additionally, the application provides an API to upload CSV or JSON files containing multiple employee records; the server automatically parses the data and inserts it into the database.

For this application, we designed three separate workloads: (i) a normal workload, where the client sends a POST request to upload a data file and the server processes each record line by line; (ii) a stress workload, where the client continuously sends multiple PUT requests to add new data, aiming to stress-test the server; and (iii) a concurrent workload, where multiple threads on the client side send PUT requests simultaneously to simulate multiple users.

Maven. Maven represents typical Java development activities. By compiling and testing Java projects, it simulates the operations developers perform in real-world software projects, providing insights into system behavior under practical development workloads. Moreover, Maven generates a large volume of detailed logs, which offer valuable information about its internal execution. In this context, Maven itself is the application under test, while different Java projects serve as inputs, each corresponding to a distinct workload.

We divided the Maven workloads into two categories: (i) mvn compile workloads, using five GitHub repositories with diverse objectives and sizes, and (ii) mvn test workloads, which include both the compile and test phases, using five distinct Java JSON libraries. The complete list of projects and libraries is presented in Table 1.

3.2 Data Collection

We collected a variety of metrics from multiple layers, including the operating system, internal hardware, network, application-specific sources, and so on. The type and scope of these metrics varied depending on the workload under test. Metrics were recorded at

Table 1: Projects and Libraries Used in Maven Workloads

Library	# Commits	Version	Type of workload
jitwatch	1062	1.4.9	Compile
litemall	1186	0.1.0	Compile
clojure	3547	1.13.0	Compile
Activiti	11081	8.7.0	Compile
languagetool	77935	6.6	Compile
JSON-java	1280	20240303	Compile & Test
esson	49	1.1.1	Compile & Test
json-simple	34	1.1.1	Compile & Test
johnzon	1067	2.0.2	Compile & Test
jackson-databind	7833	2.19.0	Compile & Test

intervals of either 100ms or 1s, depending on the experiment. A summary of all collected metrics is provided in Table 2.

Table 2: Collected metrics by workload type

Source Layer	Collected Metrics	Workload Type
Hardware	CPU Utilization	All
	Memory Utilization	All
	AS CPU Utilization	DT, CRA
	AS Memory Utilization	DT, CRA
	#Reads on disk	MV, SW, HR
	#Writes on disk	MV, SW, HR
Application Server	#Req. by Method (X)	DT, CRA
	#Req. by Response (Y)	DT, CRA
	#FFDC Exceptions	DT
Network	#Network Packets	DT, CRA
	Total Length of Packets	DT, CRA
	#HTTP Requests	DT, CRA
Database	#SQL statements	CRA
Docker	#Docker logs	HR
	#Logs from (Z)	HR
	#Slow query logs	HR
	#Slow query logs from (Z)	HR
	#Non-slow query logs	HR
	#Non-slow query logs from (Z)	HR
Maven	#Maven logs	MV
	#Logs at level (W)	MV
Others	Total Energy	All
	AS Energy	DT, CRA

Note: All collected metrics are measured as counts or values per sampling interval (either 100ms or 1s, depending on the experiment).

Workload Type: indicates which experiments generate this data.

Abbreviations: AS = Application Server; SW = Synthetic Workloads; DT = DayTrader; CRA = Custom REST App; MV = Maven; HR = Hotel Reservation.

Notation: # = number of; X = any HTTP method (GET, POST, etc.); Y = any response code (200, 404, etc.); Z = any Docker container; W = log level (INFO, WARNING, etc.).

For energy consumption, runtime measurements are obtained via the RAPL interface [24] through Linux's powercap subsystem.

⁴<https://github.com/delimitrou/DeathStarBench/tree/master/hotelReservation>

RAPL provides accurate and detailed energy monitoring of the CPU and internal components without requiring external devices or elevated privileges. Compared to statistical estimation models, RAPL delivers higher precision, making it particularly suitable for monitoring software energy consumption.

3.3 Data Analysis

To answer the research question, we analyzed the relationship between energy consumption and each of the collected metrics individually. Since the data consisted of numerical time series, we used the Pearson correlation coefficient to quantify both the strength and direction of the associations between variables. We also calculated the p-value to test the statistical significance of the correlations, ensuring that observed relationships were not due to random variation.

For each workload, the correlation coefficient was calculated independently for each experimental run. The results were then averaged across all runs to account for variability and reduce the impact of occasional outliers, providing a more stable and representative measure of the relationship between energy consumption and the metrics (see Section 4.1.1 for details).

We acknowledge that analyzing one-to-one relationships between energy consumption and individual metrics may not capture the full picture, as energy usage is influenced by many interacting factors. However, this approach was chosen for two reasons: (i) it provides a clear, interpretable first step toward understanding how single metrics reflect energy consumption, and (ii) given the time and scope limitations of this study, it offers a practical method to establish baseline insights before moving to more complex multi-variate analysis.

3.4 Experimental Infrastructure

All experiments were conducted on a Dell OptiPlex 5050 workstation equipped with an Intel Core i3-6100 processor, running Ubuntu 22.04.4 LTS with Linux kernel 5.15.0-119-generic.

The software infrastructure varies depending on the application under test. The DayTrader benchmark was connected to a DB2 database and hosted on Open Liberty 24.0.0.1. Otherwise, the Hotel Reservation application was deployed using Docker v27.5.1 and Docker Compose v2.32. The Custom REST App was hosted on Apache Tomcat 11 and connected to MySQL Server version 8.0.33-0ubuntu0.22.04.2. For the Maven experiments, Apache Maven 3.9.6, together with Java 21 2023-09-19 LTS, was used to compile and test Java projects.

Finally, all JMX files were executed using Apache JMeter 5.6.3.

4 Results

This section presents the outcomes of our experiments, focusing on the Pearson correlation between collected metrics and energy consumption across different workloads. We begin with synthetic workload experiments, where we analyze data distribution and the impact of sampling intervals to validate the consistency of our methodology. We then present the main correlation results, organized by workload type as defined in Section 3.1.

4.1 Validation of Measurement Methodology

4.1.1 Energy Data Distribution and Outliers. Figures 2 and 3 illustrate the distribution of average energy consumption for synthetic workloads across 30 iterations.

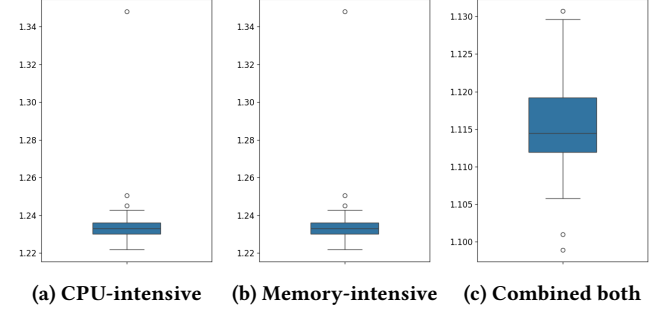


Figure 2: Distribution of average energy consumption for synthetic workloads over 30 iterations (100ms interval).

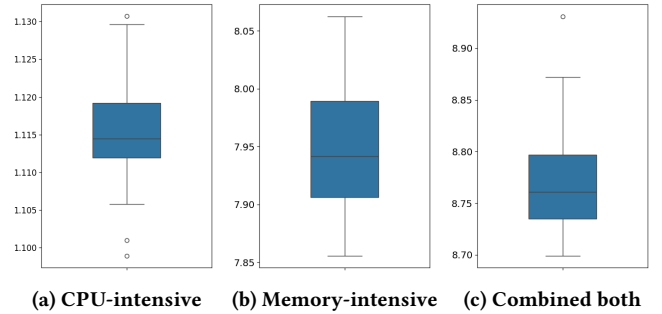


Figure 3: Distribution of average energy consumption for synthetic workloads over 30 iterations (1s interval).

In general, the energy measurements are well distributed across all runs, showing low standard deviation for both sampling intervals. For the 100ms interval, the standard deviation of average energy was 0.021 Joules for CPU-intensive, 0.008 J for Memory-intensive, and 0.007 J for the combined workload; for the 1s interval, it was 0.077J for CPU-intensive, 0.053J for Memory-intensive, and 0.055 J for the combined workload. These results demonstrate that the collected data can be reliably used for correlation analysis.

Average energy values for some runs were outside of the typical range, which can have a disproportionate impact on correlation analyses. In order to counteract this, we calculated the Pearson correlation coefficient for each run separately before averaging the results across all runs. This approach maintains the interpretability of the correlations while reducing the effect of extreme values.

4.1.2 Sampling Interval Consistency. We evaluated whether the choice of sampling interval significantly affects the precision of our statistical analyses. We initially considered that a higher data collection rate, such as 100ms, could introduce measurement overheads. At the same time, a higher rate might capture energy changes more

Table 3: Pearson correlations for collected metrics and energy consumption for synthetic workloads.

	100ms interval			1s interval		
	(i)	(ii)	(iii)	(i)	(ii)	(iii)
Collected metrics	r	r	r	r	r	r
CPU Utilization	0.489*	0.582*	0.452*	0.547*	0.739*	0.505*
#Reads on disk	0.249*	0.086	0.026	0.247*	0.060	-0.068
#Writes on disk	0.462*	0.132*	0.016	0.534*	0.129	-0.076
Memory Utilization	-0.092*	0.144*	-0.007	-0.122	0.153*	-0.085

(i) CPU-intensive; (ii) Memory-intensive; (iii) Combined tasks

 r : The Pearson correlation coefficient* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

accurately, reflecting the execution of finer-grained CPU instructions. However, Table 3, which summarizes the Pearson correlation coefficients and p-values for all synthetic workload experiments, shows no meaningful difference between the 100ms and 1s intervals. Based on these findings, we adopted a 1s sampling interval for all realistic workload experiments, with the exception of Maven, which retained a 100ms interval due to its millisecond-level log generation.

4.2 Correlations

We now turn to the main results of our study. This section presents the correlations between collected metrics and energy consumption, reporting only statistically significant relationships with Pearson's correlation coefficient satisfying $|r| > 0.3$ (moderate or stronger) and $p < 0.05$. For transparency, the full set of correlation values, including non-significant results, is provided in the supplementary material.

4.2.1 Synthetic workloads. We investigated correlations between energy consumption and test metrics for synthetic benchmarks under both 100ms and 1s sampling intervals, as shown in Table 3.

Across all experiments, CPU utilization showed a significant and consistent correlation with energy consumption, with correlation coefficients ranging from $r = 0.452$ to $r = 0.739$ and $p < 0.001$, regardless of the sampling interval. The number of writes on disk also correlated positively with energy consumption, but only in CPU-intensive workloads, where correlations were observed at both 100ms ($r = 0.462$, $p < 0.001$) and 1s ($r = 0.534$, $p < 0.001$) intervals.

In contrast, even in memory-intensive workloads, memory utilization and disk reads did not exhibit significant correlations with energy consumption.

4.2.2 Realistic workloads.

DayTrader. We observed that none of the collected metrics consistently correlated with energy consumption across both DayTrader7 and DayTrader10 (Table 4).

In DayTrader7, several network-related metrics showed moderate negative correlations with energy consumption, including the number of network packets ($r = -0.335$, $p = 0.001$) and the total packet length ($r = -0.321$, $p = 0.001$). Similarly, the number of HTTP requests ($r = -0.315$, $p = 0.001$), requests with method GET ($r = -0.313$, $p = 0.001$), and requests with response code 200

($r = -0.316$, $p = 0.001$) also exhibited moderate negative correlations.

In contrast, with DayTrader10, the only statistically significant correlation was observed for application server memory usage, which showed a moderate positive correlation with energy consumption ($r = 0.432$, $p = 0.012$).

Table 4: Significant Pearson correlations between collected metrics and energy consumption for DayTrader.

Collected metrics	DayTrader 7	DayTrader 10
	r	r
#HTTP Requests	-0.314**	-0.25*
#Req. by Method GET	-0.313**	-0.25*
#Req. by Response 200	-0.316**	-0.251*
#Network Packets	-0.335**	-0.242*
Total Length of Packets	-0.321**	-0.241*
AS Memory Utilization	0.243	0.432*

 r : The Pearson correlation coefficient* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Hotel Reservation. This application is composed of multiple containers, from which we collected up to 24 metrics during the experiment. However, as shown in Table 5, only four metrics exhibited correlations with energy consumption.

Two metrics showed statistically significant correlations: the number of Docker logs ($r = 0.592$, $p < 0.001$) and the number of slow query logs ($r = 0.716$, $p < 0.001$). At the container level, logs from the reservation container showed the strongest correlation with energy consumption ($r = 0.710$, $p < 0.001$). Separately, the number of slow query logs from the reservation container also exhibited a strong correlation ($r = 0.716$, $p < 0.001$), identical to the number of slow query logs above, indicating that all slow queries originated from this container.

Custom REST App. The number of SQL statements consistently showed a moderate positive correlation with energy consumption across all experiments, with r ranging from 0.366 to 0.460 and p values between < 0.001 and 0.006, as summarized in Table 6.

For the stress workload (experiment (ii)) and the concurrent workload (experiment (iii)), results were similar. In both cases, the

Table 5: Significant Pearson correlations between collected metrics and energy consumption for HotelReservation.

Collected metrics	r
#Docker logs	0.592***
#Logs from reservation	0.710***
#Slow query logs	0.716***
#Slow query logs from reservation	0.716***

r : The Pearson correlation coefficient
 $*p < 0.05$, $**p < 0.01$, $***p < 0.001$

number of HTTP requests, requests with method PUT, requests with response code 200, network packets, and total packet length all exhibited moderate correlations with energy consumption ($r > 0.4$, $p \leq 0.005$). Notably, the number of requests with method PUT and the number of requests with response code 200 were identical to the total number of HTTP requests in both experiments ($r = 0.423$, $p = 0.005$ for experiment (ii); $r = 0.461$, $p < 0.001$ for experiment (iii)).

In the normal workload (experiment (i)), CPU utilization showed the strongest correlation with energy consumption ($r = 0.717$, $p < 0.001$). However, in experiments (ii) and (iii), CPU utilization did not reach statistical significance despite moderate correlation values ($r > 0.4$, $p > 0.05$). Similarly, application server memory utilization showed moderate correlations in experiments (i) and (iii), but was not statistically significant.

Table 6: Significant Pearson correlations between collected metrics and energy consumption for Custom REST App.

	Exp. (i)	Exp. (ii)	Exp. (iii)
Collected metrics	r	r	r
CPU Utilization	0.717***	0.438	0.668
#HTTP Requests	0.030	0.423**	0.461***
#Req. by Method PUT	-	0.423**	0.461***
#Req. by Response 200	0.030	0.423**	0.461***
#SQL statements	0.366***	0.421**	0.460***
#Network Packets	-0.096	0.425**	0.449***
Total Length of Packets	-0.10	0.425**	0.454***

(i) Normal workload; (ii) Stress workload; (iii) Concurrent workload
 r : The Pearson correlation coefficient
 $*p < 0.05$, $**p < 0.01$, $***p < 0.001$

Maven. Our initial observations show that most collected metrics, including WARNING-level logs, CPU utilization, and memory utilization, did not exhibit significant correlations with energy consumption across the Maven projects. As summarized in Table 7, only two types of metrics, Maven log activity and disk reads, yielded statistically significant results.

For Maven logs, significant correlations were observed in two projects. In *activiti*, both the number of INFO-level logs and the total number of Maven logs showed moderate negative correlations with energy consumption ($r < -0.5$, $p < 0.001$), while in *jackson*, the same metrics exhibited moderate positive correlations ($r > 0.3$, $p < 0.05$).

Disk read counts also showed statistically significant positive correlations with energy consumption in nearly all projects, with the exception of *activiti*, *jackson*, and *johnzon*. Across the remaining projects, r values ranged from 0.306 to 0.514 with $p \leq 0.03$.

In addition to these metrics, CPU utilization showed moderate to strong positive correlations in several projects, including *jitwatch* ($r = 0.339$), *litemall* ($r = 0.326$), *esson* ($r = 0.757$), *JSON-java* ($r = 0.513$), and *JSON-simple* ($r = 0.622$). However, none of these results reached statistical significance ($p > 0.05$). Similarly, Memory utilization exhibited positive correlations but without statistical significance in *jitwatch* ($r = 0.418$), *litemall* ($r = 0.338$), *JSON-java* ($r = 0.520$), and *JSON-simple* ($r = 0.641$). In *esson*, memory utilization showed a stronger correlation ($r = 0.790$), with a p -value close to significance ($p = 0.053$).

5 Discussion

This section interprets the results presented in Section 4, answering our research questions regarding the connection between software-level metrics and energy consumption. We highlight the most reliable predictors of energy consumption, discuss the consistency of results across workloads, and reflect on the implications and limitations of our research.

5.1 Application-Specific Findings

5.1.1 Consistency across synthetic workloads. Our experiments with synthetic workloads demonstrate that CPU utilization consistently serves as a strong predictor of energy consumption for these workloads, with correlation coefficients exceeding 0.45 across all experiments. This aligns with prior empirical findings by Stefanek et al. [25], which showed that CPU utilization can provide reliable estimates of energy consumption. Disk write activity also exhibits a correlation with energy usage, though this effect is primarily observed in CPU-intensive workloads. This pattern likely reflects the I/O-intensive behavior of the tested workloads, where operations such as sorting, encryption, and compression generate frequent temporary files. Prior research by Silva et al. [6] has similarly shown that I/O operations significantly impact power and, consequently, overall energy consumption. In contrast, Memory utilization does not show a significant correlation with energy consumption, even under memory-intensive workloads. This suggests that the energy cost of memory access is relatively low compared to that of CPU computations and disk operations, meaning that variations in memory utilization alone are insufficient to produce a measurable impact on overall energy consumption.

5.1.2 Variability across realistic workloads.

DayTrader and unpredictable workload effects. For *DayTrader*, network-related metrics exhibited a moderate negative correlation with energy consumption in *DayTrader7*, whereas application server memory usage showed a positive correlation in *DayTrader10*. Notably, no metric maintained a consistent correlation across both versions of *DayTrader*.

We analyzed the Pearson correlation results across all 15 runs to gain a better understanding of this behavior, and we observed the variability. For example, in run 12, the number of HTTP requests correlated strongly and negatively with energy consumption

Table 7: Significant Pearson correlations between collected metrics and energy consumption for Maven projects.

	activiti	clojure	jitwatch	languagetool	litemall	esson	jackson	johnzon	JSON-Java	JSON-Simple
Collected data	r	r	r	r	r	r	r	r	r	r
#Maven logs	-0.507***	-0.183	-0.522	-0.116	-0.301	0.113	0.315*	-0.129	<0.001	-0.116
#Logs at level INFO	-0.537***	-0.241	-0.344	-0.116	-0.311	0.113	0.318*	-0.111	0.005	-0.116
#Reads on disk	0.199	0.394**	0.432*	0.306*	0.430*	0.435**	0.260	0.238	0.348*	0.514**

r : The Pearson correlation coefficient

* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

($r = -0.792$, $p < 0.001$), while in run 13, the correlation was positive ($r = 0.587$, $p < 0.001$). The Random Controller of JMeter, which generates unpredictable request patterns with varying computational costs, is probably the cause of this variability. These findings also highlight a drawback of our experimental setup: uncontrolled workload variability can obscure consistent metric-energy relationships. Future experiments, to better isolate the impact of application-layer activities on energy consumption, should use more controlled and repeatable workloads.

Hotel Reservation and bottleneck service effects. In the Hotel Reservation experiments, metrics extracted from software logs proved particularly informative. Both the number of Docker logs and the number of slow queries showed a strong correlation with energy consumption, with the reservation container emerging as the main contributor. This suggests that in microservice-based applications, overall energy consumption can be influenced by a single bottleneck service. In our experiment, which simulated a typical hotel reservation workflow encompassing hotel search, recommendation request, login, and reservation, at least eight containers were active. Nevertheless, only the reservation container demonstrated a significant correlation with energy consumption, indicating that it carried a substantially heavier workload than the others and thus acted as the system bottleneck.

These results highlight the potential of software-specific logs to offer valuable insights that go beyond hardware-level metrics. Even so, the fact that these results are based on a relatively simple workload may restrict how broadly they may be applied. Further studies involving more complex workloads and larger-scale deployments are necessary to figure out whether similar correlations remain in practical contexts.

Custom REST App and workload impact on energy consumption. Database activity and energy consumption are consistently correlated, as shown by the Custom REST App experiments. In particular, the number of SQL statements shows a moderate correlation across all workloads. This stability indicates that SQL queries constitute a reliable metric for estimating energy consumption, as each query corresponds to predictable database operations and associated CPU activity.

On the other hand, CPU utilization correlates with energy consumption only under the normal workload (experiment (i)). In contrast, the total packet length, the number of network packets, HTTP requests, PUT requests, and requests with a 200 response code exhibit similar correlations under both the stress (experiment (ii)) and concurrent workloads (experiment (iii)). These results highlight

the role of workload intensity in determining the key predictors of energy consumption.

Maven and variable correlations across projects. For Maven, the results are divergent, exhibiting limited and inconsistent correlations across projects. Logging metrics, including the number of Maven logs and INFO-level logs, show significant correlations within the two projects, but in opposite directions. In *activiti*, logging metrics are negatively correlated with energy consumption, whereas in *jackson* they are positively correlated, indicating that log volume alone is not a reliable predictor of energy consumption.

However, disk reads show a consistent correlation with energy consumption across most projects, suggesting a stronger relationship between I/O activity and energy usage. CPU and memory utilization occasionally exhibit strong correlations, but these do not reach statistical significance, suggesting the variability in their predictability for this type of workload.

5.2 Cross-Application Analysis

CPU utilization and SQL statements were the most reliable correlates of energy consumption across applications. Additionally, disk activity and energy usage were consistently correlated, with writes being more relevant in synthetic workload experiments and reads in Maven workloads. In contrast, because their correlations differed based on workload type and system complexity, HTTP requests and general log counts turned out to be less reliable.

These observations imply that hardware-level metrics (like CPU and disk) and structured software logs (like SQL queries) provide stronger indicators of energy consumption than unstructured activity logs (such as HTTP or Maven logs). Furthermore, workload characteristics and application structure have a strong influence on the predictive ability of a given metric. Controlled or homogeneous workloads tend to produce consistent correlations, whereas heterogeneous workloads, like those in *DayTrader*, can obscure relationships between metrics and energy consumption. Similarly, in microservice-based applications, energy consumption may be dominated by a single bottleneck service, demonstrating the need for software-level insights in addition to hardware metrics.

Answer to RQ: Can software energy consumption be estimated using software traces and logs?

Our findings indicate that certain metrics can be used to estimate energy consumption. However, the accuracy of these estimates varies based on the type of metric, workload characteristics,

and application structure. Hardware-level metrics, such as CPU utilization and disk activity, provide consistent indicators of energy consumption. In the meantime, structured software logs, such as SQL statements or service-specific logs, also show significant potential for estimating energy consumption. On the other hand, unstructured logs, such as general HTTP or Maven logs, are less reliable. In summary, the most effective approach for estimating energy consumption and enabling precise energy prediction models across applications is to combine hardware and structured software metrics, with consideration of workload and application-specific factors.

6 Limitations

Several limitations in this study should be considered when interpreting the results and may be addressed in future research.

The first limitation is that all experiments were conducted on a single machine (see details in 3.4). This setup may limit the generalizability of the results, as platforms with different performance characteristics could consume energy differently. Verifying the findings across multiple platforms would strengthen the conclusions.

The second limitation concerns the range of workloads used in the experiments. Although we aimed to maximize workload diversity, the sample size remains limited compared to the actual number. Expanding the number and diversity of workloads could provide more robust insights into how different application behaviors affect energy consumption.

Finally, the third limitation relates to the scope of the analysis. This study focused on correlations between individual metrics and energy usage rather than exploring multivariate relationships. Future work could extend the analysis to include deeper multivariate studies to examine potential interactions among metrics and their combined impact on energy consumption.

7 Conclusion

This study investigated whether software energy consumption can be estimated based on software traces and logs across diverse applications and workloads. Through several experiments on synthetic workloads and realistic applications, including DayTrader, Hotel Reservation, a Custom REST App, and Maven, we analyzed the predictive ability of both hardware-level metrics and software-specific logs.

Our results demonstrate that hardware metrics, especially CPU utilization and disk activity, consistently correlate with energy consumption across applications. Structured software logs, such as SQL statements or Docker container logs (which capture the activity of individual microservices), also reveal their potential in estimating energy consumption. This is particularly true, especially when energy use is dominated by a single service or component. In contrast, unstructured logs, including general HTTP requests or Maven logs, exhibit unstable correlations, emphasizing the limitations of relying solely on unstructured traces for energy monitoring.

These findings indicate several important insights for energy-aware software design and monitoring. The most reliable approach for estimating energy consumption is to combine both hardware metrics and structured software traces. Furthermore, workload characteristics and application structure strongly influence the

predictive power of different metrics, emphasizing the necessity for energy prediction approaches that specifically take workload type, intensity, and execution patterns into account.

Our initial study provides a solid empirical basis for future development. Future work, which builds on these findings, should expand these findings through larger-scale experiments, more complex workloads, and diverse application architectures. In addition, developing tools and frameworks that integrate hardware and software metrics for real-time energy monitoring could facilitate energy-efficient software development and deployment. Last but not least, the existing lack of energy-relevant logs offers a chance to design energy-aware logging mechanisms that may improve software energy consumption prediction and proactive management.

Acknowledgments

The research leading to this publication is supported by the Communauté de communes de Lacq-Orthez (CCLO, France). Additional support was provided by the RESILINK project, funded under PRIMA S2 2021 (ID 1707), which is supported by the European Union.

References

- [1] Amruta Ambre and Narendra Shekhar. 2015. Insider Threat Detection Using Log Analysis and Event Correlation. *Procedia Computer Science* 45 (2015), 436–445. doi:10.1016/j.procs.2015.03.175 International Conference on Advanced Computing Technologies and Applications (ICACTA).
- [2] Merve Astekin, Selim Özcan, and Hasan Sözer. 2019. Incremental Analysis of Large-Scale System Logs for Anomaly Detection. In *2019 IEEE International Conference on Big Data (Big Data)*. 2119–2127. doi:10.1109/BigData47090.2019.9006593
- [3] Seth Ayers, Sara Ballan, Vanessa Gray, and Rosie McDonald. 2023. Measuring the Emissions and Energy Footprint of the ICT Sector: Implications for Climate Action. <http://documents.worldbank.org/curated/en/099121223165540890>. (English).
- [4] Lotfi Belkhir and Ahmed Elmeligi. 2018. Assessing ICT global emissions footprint: Trends to 2040 & recommendations. *Journal of Cleaner Production* 177 (2018), 448–463. doi:10.1016/j.jclepro.2017.12.239
- [5] Andre Brandao and Petia Georgieva. 2020. Log Files Analysis For Network Intrusion Detection. In *2020 IEEE 10th International Conference on Intelligent Systems (IS)*. 328–333. doi:10.1109/IS48319.2020.9199976
- [6] Rafael Ferreira da Silva, Henri Casanova, Anne-Cécile Orgerie, Ryan Tanaka, Ewa Deelman, and Frédéric Suter. 2020. Characterizing, Modeling, and Accurately Simulating Power and Energy Consumption of I/O-intensive Scientific Workflows. *Journal of Computational Science* 44 (2020), 101157. doi:10.1016/j.jocs.2020.101157
- [7] Guillaume Fieni, Daniel Romero Acero, Pierre Rust, and Romain Rouvoy. 2024. PowerAPI: A Python framework for building software-defined power meters. *Journal of Open Source Software* 9, 98 (June 2024), 6670. doi:10.21105/joss.06670
- [8] Qiang Fu, Jian-Guang Lou, Qingwei Lin, Rui Ding, Dongmei Zhang, and Tao Xie. 2013. Contextual analysis of program logs for understanding system behaviors. In *Proceedings of the 10th Working Conference on Mining Software Repositories (San Francisco, CA, USA) (MSR '13)*. IEEE Press, 397–400.
- [9] Shilin He, Jieming Zhu, Pinjia He, and Michael R. Lyu. 2016. Experience Report: System Log Analysis for Anomaly Detection. In *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*. 207–218. doi:10.1109/ISSRE.2016.21
- [10] Jens Malmelin, Nina Lövehagen, Pernilla Bergmark, and Dag Lundén. 2024. ICT sector electricity consumption and greenhouse gas emissions – 2020 outcome. *Telecommunications Policy* 48, 3 (2024), 102701. doi:10.1016/j.telpol.2023.102701
- [11] Leonardo Mariani and Fabrizio Pastore. 2008. Automated Identification of Failure Causes in System Logs. In *2008 19th International Symposium on Software Reliability Engineering (ISSRE)*. 117–126. doi:10.1109/ISSRE.2008.48
- [12] Matias Martinez, Silverio Martinez-Fernández, and Xavier Franch. 2025. The Sustainability Face of Automated Program Repair Tools. *ACM Trans. Softw. Eng. Methodol.* (June 2025). doi:10.1145/3744900 Just Accepted.
- [13] Karthik Nagaraj, Charles Killian, and Jennifer Neville. 2012. Structured Comparative Analysis of Systems Logs to Diagnose Performance Problems. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. USENIX Association, San Jose, CA, 353–366.
- [14] Adel Noureddine. 2022. PowerJoular and JoularJX: Multi-Platform Software Power Monitoring Tools. In *18th International Conference on Intelligent Environments (IE2022)*. Biarritz, France.

- [15] Adel Nouredine. 2022. PowerJoular and JoularJX: Multi-Platform Software Power Monitoring Tools. In *2022 18th International Conference on Intelligent Environments (IE)*. 1–4. doi:10.1109/IE54923.2022.9826760
- [16] Adel Nouredine, Matias Martinez, Houssam Kanso, and Noëlle Bru. 2022. *Is Well-Tested Code More Energy Efficient?* doi:10.3233/AISE220044
- [17] Wellington Oliveira, Renato Oliveira, Fernando Castor, Gustavo Pinto, and João Paulo Fernandes. 2021. Improving energy-efficiency by recommending Java collections. *Empirical Softw. Engg.* 26, 3 (May 2021), 45 pages. doi:10.1007/s10664-021-09950-y
- [18] Zakaria Ournani, Mohammed Chakib Belgaid, Romain Rouvoy, Pierre Rust, and Joel Penhoat. 2021. Evaluating the Impact of Java Virtual Machines on Energy Consumption. In *15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. Bari, Italy. <https://inria.hal.science/hal-03275286>
- [19] Zakaria Ournani, Romain Rouvoy, Pierre Rust, and Joel Penhoat. 2020. On Reducing the Energy Consumption of Software: From Hurdles to Requirements. In *Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (Bari, Italy) (ESEM '20)*. ACM, NY, USA, Article 14, 12 pages. doi:10.1145/3382494.3410678
- [20] Zakaria Ournani, Romain Rouvoy, Pierre Rust, and Joel Penhoat. 2021. Comparing the Energy Consumption of Java I/O Libraries and Methods. In *Proceedings of the 37th International Conference on Software Maintenance and Evolution (ICSME) (Proceedings of the 37th International Conference on Software Maintenance and Evolution (ICSME))*. Luxembourg / Virtual, Luxembourg. <https://inria.hal.science/hal-03269129>
- [21] Antonio Pecchia and Stefano Russo. 2012. Detection of Software Failures through Event Logs: An Experimental Study. In *2012 IEEE 23rd International Symposium on Software Reliability Engineering*. 31–40. doi:10.1109/ISSRE.2012.24
- [22] Gustavo Pinto, Fernando Castor, and Yu David Liu. 2014. Mining questions about software energy consumption. In *Proceedings of the 11th Working Conference on Mining Software Repositories (Hyderabad, India) (MSR 2014)*. Association for Computing Machinery, New York, NY, USA, 22–31. doi:10.1145/2597073.2597110
- [23] Ajitha Rajan, Adel Nouredine, and Panagiotis Stratis. 2016. A Study on the Influence of Software and Hardware Features on Program Energy. In *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (Ciudad Real, Spain) (ESEM '16)*. ACM, NY, USA, Article 37, 10 pages. doi:10.1145/2961111.2962593
- [24] Efraim Rotem, Alon Naveh, Avinash Ananthakrishnan, Eliezer Weissmann, and Doron Rajwan. 2012. Power-Management Architecture of the Intel Microarchitecture Code-Named Sandy Bridge. *IEEE Micro* 32, 2 (2012), 20–27. doi:10.1109/MM.2012.12
- [25] Anton Stefanek, Uli Harder, and Jeremy T. Bradley. 2013. Energy Consumption in the Office. In *Computer Performance Engineering*, Mirco Tribastone and Stephen Gilmore (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 224–236.
- [26] Xiwei Xu, Liming Zhu, Ingo Weber, Len Bass, and Daniel Sun. 2014. POD-Diagnosis: Error Diagnosis of Sporadic Operations on Cloud Applications. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. 252–263. doi:10.1109/DSN.2014.94
- [27] Ding Yuan, Haohui Mai, Weiwei Xiong, Lin Tan, Yuanyuan Zhou, and Shankar Pasupathy. 2010. SherLog: error diagnosis by connecting clues from run-time logs. In *Proceedings of the Fifteenth International Conference on Architectural Support for Programming Languages and Operating Systems (Pittsburgh, Pennsylvania, USA) (ASPLOS XV)*. ACM, New York, NY, USA, 143–154. doi:10.1145/1736020.1736038
- [28] Wei Yuan, Shan Lu, Hailong Sun, and Xudong Liu. 2020. How are distributed bugs diagnosed and fixed through system logs? *Information and Software Technology* 119 (2020), 106234. doi:10.1016/j.infsof.2019.106234

A Experimental Data

All datasets and results generated during the experiments of this study are publicly available in the following repository:
<https://doi.org/10.5281/zenodo.17469762>